18

Os prompts da imersão

Todos os prompts usados ao vivo, na ordem em que aparecem na construção do projeto — da entrevista de requisitos ao RAG. Copie, ajuste os campos marcados e cole no Claude Code (ou no OpenCode).

COMO USAR ESTE KIT

Cada prompt depende do artefato gerado pelo anterior. Siga a ordem no primeiro projeto — depois reaproveite só os que fizerem sentido.

Trechos com [fundo amarelo] são campos para você preencher. Todo o resto pode ser colado como está.

Os prompts citam **SPEC.md**, **CLAUDE.md** e **PLAN.md** na raiz do projeto. Se você usa OpenCode, leia **AGENTS.md** onde aparecer CLAUDE.md.

Índice

#	PROMPT	GERA
1	Criar o SPEC.md via entrevista de requisitos	SPEC.md + artefatos de SDD
2	Template do SPEC.md	Esqueleto para preencher à mão
3	Gerar o Context File a partir do SPEC	CLAUDE.md / AGENTS.md
4	Gerar o scaffold completo do projeto	Pastas, configs, /health, smoke test
5	Gerar arquivos de exclusão de contexto	.claudeignore / .gitignore
6	Slash command de nova implementação	/nova-implementacao
7	Slash commands /review, /implementar e /entrega	3 comandos do fluxo
8	Hooks específicos por agent	Hooks + scripts de bloqueio
9	Executar uma fase do PLAN com orquestrador + reviewer	Tasks implementadas e revisadas
10	Orquestrador relatar qual agent está ativo	Regra de transparência

11	Sistema de RAG com captura de aprendizados	rag.db + scripts + hooks

1 Criar o SPEC.md via entrevista de requisitos

QUANDO USAR No início de um projeto novo, antes de qualquer código. A IA pergunta, você responde — uma pergunta por vez.

PROMPT

```
Você vai me ajudar a criar o SPEC.md e todos os demais arquivos de especificação utilizando os fundamentos do Spec Driven Development. No mínimo você deve criar Constitution, Design, Spec, Plan, Tasks, Implement do meu projeto. Mas não quero escrever o spec manualmente — quero que você conduza uma entrevista de requisitos comigo.

Fase 1 — Entrevista:
Faça perguntas para entender o projeto a fundo. Uma pergunta por vez. Cubra obrigatoriamente: problema real que resolve, quem são os usuários, funcionalidades essenciais vs. nice-to-have, módulos e suas responsabilidades, stack técnica, constraints e o que está fora do escopo. Quando tiver informação suficiente, avise que está pronto para gerar o SPEC.

Fase 2 — Geração do SPEC.md:
Quando eu confirmar, gere um arquivo SPEC.md e todos os outros arquivos necessários de Spec Driven Development seguindo fielmente os fundamentos do SDD com esta estrutura:

# SPEC.md

## Problema
## Usuários
## Funcionalidades
### Essenciais
### Fora do escopo
## Módulos
## Stack
## Constraints técnicas
## Critérios de aceitação

Não invente informações que não surgiram na entrevista. Se algo ficou ambíguo, registre como decisão em aberto.
```

2 Template do SPEC.md

QUANDO USAR Se preferir escrever o SPEC você mesmo. Não é um prompt: é o esqueleto que o prompt 1 gera, para preencher à mão.

TEMPLATE

```
# SPEC.md

## Problema
Descreva o problema que o projeto resolve em 2-3 frases.
```

```
Quem sofre com isso hoje? Por que é importante?

## Usuários
Quem vai usar? Qual o perfil?

## Funcionalidades
### Essenciais
- Funcionalidade 1
- Funcionalidade 2
- Funcionalidade 3

### Fora do escopo
- O que NÃO será feito nesta versão

## Módulos
Módulo 1: [nome] - [responsabilidade]
Módulo 2: [nome] - [responsabilidade]
Módulo 3: [nome] - [responsabilidade]

## Stack
Frontend: [framework]
Backend: [framework]
Database: [banco]

## Constraints técnicas
- Limitações de tempo, infra, APIs
- Dependências externas conhecidas

## Critérios de aceitação
- [ ] Critério 1
- [ ] Critério 2
- [ ] Critério 3
```

3 Gerar o Context File a partir do SPEC

QUANDO USAR Logo depois do SPEC. O Context File é o que o agent lê em toda sessão. Claude Code → CLAUDE.md · OpenCode → AGENTS.md.

PROMPT

```
Leia o SPEC.md abaixo e gere o Context File do projeto.
Claude Code → salve como CLAUDE.md · OpenCode → salve como AGENTS.md (Se ainda não existirem, se existir, apenas atualize)

Leia o arquivo SPEC.md na raiz do projeto.

Gere o arquivo exatamente nesta estrutura, sem adicionar nem remover seções:

---

# [NOME DO PROJETO]
> [1 frase: o que o sistema faz e qual problema resolve]
```

```
## Stack
| Camada | Tecnologia |
|--------|------------|
[preencha com a stack do SPEC]

## Estrutura de pastas
[árvore de pastas do projeto com comentário em cada pasta]

## Como rodar localmente
[comandos exatos de instalação e execução, separados por backend e frontend]

## Padrões de código
- Nomenclatura de arquivos: [regra]
- Nomenclatura de variáveis e funções: [regra]
- Estrutura de endpoints: [regra]
- Estrutura de componentes: [regra]
- Tipagem: [regra]

## TDD
- Framework backend: [nome]
- Framework frontend: [nome]
- Onde ficam os testes: [caminho]
- Regra: [cobertura mínima exigida]
- Testes críticos deste projeto:
  - [ ] [teste 1 extraído do SPEC]
  - [ ] [teste 2 extraído do SPEC]
  - [ ] [teste 3 extraído do SPEC]

## Nunca fazer
- Nunca [proibição específica deste projeto]
- Nunca [proibição específica deste projeto]
- Nunca [proibição específica deste projeto]
- Nunca [proibição específica deste projeto]
- Nunca [proibição específica deste projeto]

## Decisões em aberto
- [ ] [decisão 1 não resolvida no SPEC]
- [ ] [decisão 2 não resolvida no SPEC]

---

Preencha apenas os campos entre colchetes.
Não invente nada que não esteja no SPEC.
Se uma informação não existir no SPEC, escreva "a definir".
```

> Aqui os colchetes **não** são para você: quem preenche é a IA, a partir do SPEC. Cole o prompt como está.

4 Gerar o scaffold completo do projeto

QUANDO USAR Com SPEC, CLAUDE.md e PLAN.md prontos. Prepara o terreno — sem nenhuma regra de negócio.

PROMPT

```
Leia os arquivos SPEC.md, CLAUDE.md (ou AGENTS.md)
e PLAN.md na raiz do projeto.

Gere o scaffold completo do projeto seguindo estas regras:

1. ESTRUTURA DE PASTAS
   Crie todos os diretórios e subdiretórios definidos
   na seção de estrutura do CLAUDE.md/AGENTS.md.
   Adicione um arquivo README.md de uma linha em cada
   pasta descrevendo sua responsabilidade.

2. ARQUIVOS DE CONFIGURAÇÃO
   Crie todos os arquivos de configuração da stack:
   requirements.txt, package.json, tsconfig.json,
   tailwind.config.js, .env.example, .gitignore
   e qualquer outro necessário para a stack do projeto.
   Instale as dependências após criar os arquivos.

3. PONTO DE ENTRADA MÍNIMO
   Crie um main.py (ou index.ts, server.js — conforme a stack)
   com apenas o servidor inicializado e uma rota /health
   que retorna status 200. Nenhuma lógica de negócio.

4. SETUP DE TESTES
   Configure o framework de testes definido no CLAUDE.md:
   crie a pasta de testes espelhando a estrutura do projeto
   e um teste de smoke que verifica se o servidor sobe.

5. O QUE NÃO FAZER
   - Não implemente nenhuma lógica de negócio
   - Não crie modelos de dados
   - Não crie endpoints funcionais além de /health
   - Não crie componentes de UI além do layout base

6. CRITÉRIO DE CONCLUSÃO
   Ao terminar, execute os comandos de verificação
   definidos no CLAUDE.md e confirme que o servidor
   sobe sem erros antes de encerrar.
```

5 Gerar arquivos de exclusão de contexto

QUANDO USAR Assim que o scaffold existir (já há node_modules, build, .env). Mantém lixo e segredos fora do contexto do agent.

PROMPT

```
Analise a estrutura do meu projeto e gere os arquivos de exclusão de contexto adequados para cada
ferramenta que estou usando.

Stack do projeto: [INFORME A STACK — ex: FastAPI + Next.js + SQLite]
```

```
Ferramentas de IA: [Claude Code / OpenCode / ambos]

Para Claude Code, gere o arquivo .claudeignore com estas seções obrigatórias, nesta ordem:

# Dependências
[liste os diretórios de dependências da stack]

# Build e arquivos gerados
[liste os diretórios de build e arquivos gerados]

# Segredos e variáveis de ambiente
[liste todos os padrões de arquivos com credenciais]

# Banco de dados local
[liste arquivos de banco de dados de desenvolvimento]

# Uploads e mídia
[liste diretórios de arquivos enviados pelo usuário]

# Cache e logs
[liste diretórios de cache e arquivos de log]

Para OpenCode, gere um .gitignore reforçado cobrindo as mesmas categorias acima — pois o OpenCode
ainda não possui arquivo de exclusão próprio e depende do .gitignore.

Regra: seja conservador. Em caso de dúvida, exclua.
Nunca exclua arquivos de código fonte, testes ou documentação.
```

Atenção a segredos: o bloqueio garantido de leitura de arquivos sensíveis no Claude Code é o **permissions.deny** do settings.json (ex.: `Read(./.env)`). Use o arquivo de exclusão para reduzir ruído, e o deny para proteger credenciais.

6 Slash command de nova implementação/feature

QUANDO USAR Quando o projeto já roda e você quer um fluxo repetível para cada feature nova: entrevista → plano → aprovação → implementação → testes → relatório.

PROMPT

```
Crie um comando novo slash capaz de criar uma nova implementação/feature.

Nome do comando nova-implementacao
Esse comando deve fazer uma entrevista comigo pra saber que tipo de implementação eu desejo
construir.
Depois, isso deve criar o plano de execução, com sprint, análise de impacto no sistema, testes TDD e
apresentar o plano ao desenvolvedor para aprovação.

Se o programador aceitar, deve iniciar a implementação com comunicação clara de cada etapa que está
sendo desenvolvida.
Ao finalizar, deve rodar os testes, auto corrigir aquilo que falhou e apresentar um relatório final
```

```
do que foi implementado.

Deve criar também um plano de testes para o QA montar o ambiente de testes e testar a nova
implementação.

Crie o comando em /commands do projeto
```

7 Slash commands /review, /implementar e /entrega

QUANDO USAR Depois de criar os agents em .claude/agents/. Transforma o fluxo do PLAN em três comandos curtos.

PROMPT

```
Leia o CLAUDE.md (ou AGENTS.md), o PLAN.md
e os agents criados em .claude/agents/.

Crie três slash commands para o projeto:

1. /review
   Invoca o @code-reviewer com o SPEC.md
   e o PLAN.md como referência.
   Classifica os problemas encontrados em
   BLOQUEANTE, IMPORTANTE e SUGESTÃO.

2. /implementar [task]
   Lê a task especificada no PLAN.md,
   despacha o agent correto para aquele módulo,
   enforça TDD e dispara o @code-reviewer
   ao final automaticamente.

3. /entrega
   Roda os hooks de encerramento,
   verifica os checkboxes do PLAN.md,
   gera o relatório de próximos passos
   e confirma que o projeto está pronto
   para ser clonado por outra pessoa.

Para Claude Code: crie em .claude/commands/
Para OpenCode: crie em .opencode/commands/
```

8 Hooks específicos por agent

QUANDO USAR Quando os agents já existem e você quer que as regras deixem de ser pedido e virem bloqueio real.

PROMPT

```
Leia os agents criados em .claude/agents/
e o PLAN.md na raiz do projeto.

Para cada agent, crie os hooks específicos
que façam sentido para o seu domínio e escopo.
```

```
Siga estas regras por tipo de agent:

AGENT SOMENTE LEITURA (revisor, auditor):
- PreToolUse que bloqueia Write, Edit e Bash
  com mensagem clara de por que está bloqueando

AGENT DE IMPLEMENTAÇÃO (backend, frontend, parser):
- PostToolUse que roda os testes do módulo
  após qualquer edição de arquivo
- PreToolUse que bloqueia comandos fora do escopo
  (ex: backend não faz git push, parser não
  acessa banco de dados)
- Stop que verifica se os testes passam
  antes de encerrar

AGENT DE QA/TESTES:
- PreToolUse que permite apenas comandos
  de execução de testes (pytest, vitest, jest)
  e bloqueia qualquer outra coisa

Para Claude Code:
- Adicione a seção hooks no frontmatter
  de cada arquivo em .claude/agents/
- Crie os scripts em .claude/hooks/nome-do-agent/
- Scripts devem usar exit 2 para bloquear
  e exit 0 para permitir

Para OpenCode:
- Crie ou atualize .opencode/plugin/agent-hooks.ts
  verificando o nome do agent em cada hook

Ao terminar, demonstre um caso de uso real:
um agent tentando fazer algo fora do seu escopo
e sendo bloqueado pelo hook.
```

9 Executar uma fase do PLAN com orquestrador + reviewer

QUANDO USAR Para executar uma fase inteira do PLAN.md. Troque o número da fase e o nome do agent de implementação pelos do seu projeto.

PROMPT

```
Leia o PLAN.md e execute a Fase 2 completa
seguindo este fluxo obrigatório para cada task:

1. Despache @backend-agent para implementar a task
2. Aguarde o retorno confirmando conclusão
3. Despache @code-reviewer para revisar o que
   foi implementado — em contexto isolado
4. Aguarde o veredicto do reviewer
5. Se houver item BLOQUEANTE: corrija antes
```

```
   de avançar para a próxima task
6. Se não houver bloqueantes: avance

Não pule o step de review em nenhuma task.
O orquestrador despacha cada agent separadamente —
o reviewer nunca vê a conversa de implementação,
só o output final. É o contexto mais limpo possível.
```

10 Orquestrador relatar qual agent está ativo

QUANDO USAR Junto com o prompt 9. Sem isso, você não sabe quem está trabalhando em cada momento.

PROMPT

```
oriente o agente orquestrador e todos os outros agentes a sempre relatar no chat qual agent esta
sendo usado no momento principalmente quando eles fizerem gates entre eles
```

> Para a regra valer em todas as sessões, peça que ela seja gravada no CLAUDE.md. Exemplo do que aparece no chat: `▶ backend-dev: implementando T20…` · `▶ code-reviewer: revisando T20 (contexto isolado)…`

11 Sistema de RAG com captura de aprendizados

QUANDO USAR Em projetos longos, quando os mesmos erros e decisões começam a se repetir entre sessões. É o mais avançado do kit.

PROMPT

```
Leia o SPEC.md, o CLAUDE.md e o PLAN.md
na raiz do projeto.

Crie um sistema de RAG para este projeto
que captura aprendizados em três momentos:
ao encerrar um agent, ao encerrar a sessão
e ao iniciar um novo prompt.

O sistema deve ter:

1. SCHEMA DO BANCO
   SQLite com sqlite-vec
   Tabela knowledge com:
   id, path, content, embedding,
   category, agent, created_at

2. SCRIPT DE RESUMO (.claude/scripts/summarize.ts)
   Recebe como input o contexto a resumir
   (transcript de agent ou de sessão inteira)
   Extrai aprendizados em 4 categorias:
   - bugs resolvidos com causa raiz
```

```
   - decisões de arquitetura e o porquê
   - padrões adotados pelo time
   - o que não funcionou e por quê
   Salva em .claude/knowledge/YYYY-MM-DD-agent-slug.md

3. SCRIPT DE EMBEDDING (.claude/scripts/embed.ts)
   Lê o arquivo .md gerado
   Quebra em chunks de 200-300 tokens
   Gera embeddings com all-MiniLM-L6-v2
   Faz INSERT no rag.db com agent e category

4. SCRIPT DE BUSCA (.claude/scripts/search.ts)
   Recebe a pergunta atual e o agent atual
   Gera embedding da pergunta
   Busca top-3 chunks mais similares
   Prioriza chunks da mesma categoria do agent ativo
   Retorna o conteúdo para injetar no contexto

5. SUBAGENT STOP HOOK (async — por agent)
   Captura o aprendizado específico do agent
   que acabou de terminar uma task

6. STOP HOOK (async — por sessão)
   Captura o aprendizado da sessão inteira
   como síntese do que aconteceu

7. USERPROMPTSUBMIT HOOK
   Busca conhecimento relevante para o prompt atual
   Prioriza a categoria do agent ativo
   Injeta os chunks encontrados no contexto
```

COLA PARA FIXAR

- **1 → 3:** SPEC primeiro, Context File depois. O CLAUDE.md nasce do SPEC, nunca da imaginação.
- **4 → 5:** scaffold sem regra de negócio, e só então as exclusões de contexto.
- **6 → 7:** comandos transformam o fluxo em rotina — ninguém precisa lembrar dos passos.
- **8:** hook é regra que bloqueia; instrução no prompt é regra que se pede.
- **9 → 10:** implementação e review em agents separados, e sempre visível quem está ativo.
- **11:** RAG só quando o projeto já tem histórico para lembrar.